

科目：データベース演習

第15回：最終課題 アプリケーション構築と分析SQL

→ 2025年度は第14回に実施

講師：鈴木源吾

前回の復習

1. 最終課題用のデータベース作成と課題の説明
2. ER図, ER図とテーブルとの対応の復習
3. 最終課題データベースのER図作成

今回のトピック

1. 最終課題データベースを活用したアプリケーション構築
2. 最終課題データベースを活用した分析SQL

topic 1

最終課題データベースを活用したアプリケーション構築

Pythonによるコマンドラインアプリケーション

第8・9回の演習でコマンドラインアプリケーションを作成し，以下のパッケージを利用した

- `sqlite3`：データベースにアクセスするドライバ
- `sqlalchemy`：データベースにアクセスするフレームワーク

最終課題では，`sqlite3`を利用したアプリケーションの作成を行う

最終課題におけるアプリケーション作成

- 最終課題では、既存のデータベースを調査し、データベースを拡張する状況を想定した
- このデータベースでは、従業員や顧客が地域と関連づけられている
 - 例) 顧客（の会社）がある都市に存在している
 - 例) 従業員がある都市や国に住んでいる
- そこで、従業員や顧客と地域との関連を調べるようなアプリケーションを作成してみよう
- その結果を、地域のデータ管理方法などに活用する

サンプルプログラム15-1：検索

以下は、パリにいる顧客をすべて検索するプログラムである。

```
import sqlite3

connection = sqlite3.connect('ex_final.sqlite3')

cursor = connection.cursor()
cursor.execute("SELECT * FROM customers WHERE city = 'Paris';")

for row in cursor :
    print(row)

cursor.close()
connection.close()
```

サンプルプログラム15-1の実行結果

実行すると、以下のような2行の結果が得られる。正しく実行できるか確かめよう。

```
('PARIS', 'Paris spécialités', 'Marie Bertrand', (中略), '(1) 42.34.22.77')  
('SPECD', 'Spécialités du monde', 'Dominique Perrier', (中略), '(1) 47.55.60.20')
```

sqlite3の復習：接続

```
connection = sqlite3.connect('ex_final.sqlite3')
```

接続オブジェクトconnectionをconnect関数により作成する.

- ex_final.sqlite3というファイル名のデータベースに接続する

sqlite3の復習：実行

```
cursor = connection.cursor()  
cursor.execute("SELECT * FROM customers WHERE city = 'Paris';")
```

- カーソルオブジェクトcursorをcursor関数により生成し
 - カーソルの解説は、第8回資料参照
- カーソルオブジェクトに対して、execute関数によりSQL文を実行する

sqlite3の復習：結果取得

```
for row in cursor :  
    print(row)
```

- カーソルオブジェクトcursorがあたかもリストであるように，結果は取得できる
- rowに1行の結果が入る．rowはPythonのタプルである

sqlite3の復習：終了処理

```
cursor.close()  
connection.close()
```

- 終了処理は忘れずに、接続の解除やメモリの解放が行われる

最終課題 問4

最終課題データベースのテーブルemployeesを用いて、ロンドン（London）に住んでいる（カラムcity）従業員の名（カラムfirst_name）と姓（カラムlast_name）を順番に出力するアプリケーションを作成せよ。出力は（カラムの）並べ替えを行わずに、Pythonのタプルをそのまま出力すること。

以下に実行結果を示す

```
('Steven', 'Buchanan')  
( 'Michael', 'Suyama')  
( 'Robert', 'King')  
( 'Anne', 'Dodsworth')
```

サンプルプログラム15-2：引数・パラメータの使用

以下は、引数より都市の値を得て、その都市の顧客を求めるプログラムである

```
import sqlite3
import sys

if len(sys.argv) != 2 :
    print("argument error")
    exit()

city = sys.argv[1]
connection = sqlite3.connect('ex_final.sqlite3')
cursor = connection.cursor()
cursor.execute('SELECT * FROM customers WHERE city = ?;', (city,))

for row in cursor :
    print(row)
cursor.close()
connection.close()
```

サンプルプログラムの実行結果

コマンドラインで引数を1つ指定する必要がある。
引数を指定しないと、以下のようにエラーで終了する。

```
!python3 sample02.py  
argument error
```

引数で都市の文字列を入れると、その都市にいる顧客を求める

```
!python3 sample02.py Paris  
( 'PARIS', 'Paris spécialités', 'Marie Bertrand', (中略), '(1) 42.34.22.77' )  
( 'SPECD', 'Spécialités du monde', 'Dominique Perrier', (中略), '(1) 47.55.60.20' )
```

復習：コマンドライン引数の利用

```
if len(sys.argv) != 2 :  
    print("argument error")  
    exit()  
  
city = sys.argv[1]
```

- コマンドラインの引数は, `sys.argv`にリストとして入る
- if文で, コマンドラインの引数の数をチェックしている. 2ではない場合がエラーである. `sys.argv[0]`にはプログラム名が入るから, それに加えてもう一つの引数がないとエラーとなる

復習：パラメータの利用

```
cursor.execute('SELECT * FROM customers WHERE city = ?;', (city,))
```

- sqlite3では、SQL文の一部をパラメータ化できる
- ?の部分に、変数cityの値が入る.
- 変数cityにはコマンドライン引数で指定した値が入ったから、例えば、引数でParisを指定すれば、 "SELECT * FROM customers WHERE city = 'Paris'"が発行される

最終課題 問5

最終課題データベースのテーブルemployeesを用いて、以下のアプリケーションを作成せよ

- コマンドラインの引数で、国の文字列を受け付け、
- 引数に指定した国と同じ国（カラムcountry）に住む従業員を検索し、
- 従業員の名（カラムfirst_name）と姓（カラムlast_name）をこの順番で、
- 出力するアプリケーションを作成せよ.
- 出力は、Pythonのタプルをそのまま出力すること

最終課題 問5

以下に実行例を示す

```
!python3 DS_EX_FINAL_05.py USA  
( 'Nancy', 'Davolio' )  
( 'Andrew', 'Fuller' )  
( 'Janet', 'Leverling' )  
( 'Margaret', 'Peacock' )  
( 'Laura', 'Callahan' )
```

topic 2

最終課題データベースを活用した分析SQL

最終課題におけるSQLによる分析

- 最終課題では、既存のデータベースを調査し、データベースを拡張する状況を想定した
- このデータベースでは、顧客からの注文の情報を管理している
- 注文に関連する情報としては、例えば、以下の情報がある
 - 例) 注文における製品の単価や数
 - 例) 注文における要望された日付や出荷された日付
- そこで、一回あたりの注文金額がどの程度か、注文における出荷に遅れがないか、などの注文に関する状況を把握するために、SQLを利用して分析してみよう

分析に利用するSQL

分析に利用するSQLとして以下を学んだ

- 演算・関数
- ジョイン、外部ジョイン
- グループ化
- サブクエリ
- ウィンドウ関数

最後にこれを利用して、最終課題データベースの検索を行ってみよう

例題1

注文詳細テーブル (order_details) には、注文に含まれる製品番号 (product_idカラム)、製品の単価 (unit_priceカラム)、製品の数 (quantityカラム)、その割引率 (discount) が管理されている。これらを用いて、注文番号 (order_idカラム)、製品番号、注文と製品毎の注文金額 (一つの注文における一種類の製品に関する金額総額。結果は小数第1位を四捨五入した整数とすること。整数に変換するのにround関数を用いること。カラムの別名をorder_product_priceとせよ) の3つの組を求めるSQL文を作成せよ。ただし、結果の行数は10に限定せよ。

例題1の検索結果例

	 order_id [PK] smallint 	product_id [PK] smallint 	order_product_price double precision 
1	10248	11	168
2	10248	42	98
3	10248	72	174
4	10249	14	167
5	10249	51	1696
6	10250	41	77
7	10250	51	1261
8	10250	65	214
9	10251	22	96
10	10251	57	222

例題2

注文毎の注文金額（注文に含まれるすべての製品に関する金額総額の合計）を求めたい。注文テーブル（orders）と注文詳細テーブル（order_details）を用い、例題1の答えも参考にしつつ、注文番号、注文毎の注文金額（カラムの別名はorder_priceとせよ）の2つの組を求めるSQL文を作成せよ。ただし、結果の行数は10に限定せよ

例題2の検索結果例

	 order_id [PK] smallint 	order_price double precision 
1	10248	440
2	10249	1863
3	10250	1552
4	10251	654
5	10252	3597
6	10253	1445
7	10254	557
8	10255	2490
9	10256	518
10	10257	1119

例題3

注文テーブル（orders）には、注文のあった日付（order_dateカラム）、顧客から要望された届け日（required_dateカラム）、出荷された日付（shipped_date）を管理している。顧客から要望された届け日に間に合わなかった（出荷された日付が、顧客から要望された届け日を超えた）注文の、注文番号、顧客から要望された届け日、出荷された日付の3つの組を求めるSQL文を作成せよ。ただし、結果の行数は10に限定せよ

★注意★

- PostgreSQLでは、日付の引き算などの演算が直感通りに動くが、SQLiteでは内部的に文字列であることからうまく動作しない
- 日付型の差で日数を求めたいときは、ユリウス日を求める関数を使いユリウス日を求めて、その差をとればよい
- 参考：ユリウス日とは、紀元前4713年1月1日正午（ユリウス暦）からの経過日数

例題3の検索結果例

	 order_id [PK] smallint 	required_date date 	shipped_date date 
1	10264	1996-08-21	1996-08-23
2	10271	1996-08-29	1996-08-30
3	10280	1996-09-11	1996-09-12
4	10302	1996-10-08	1996-10-09
5	10309	1996-10-17	1996-10-23
6	10320	1996-10-17	1996-10-18
7	10380	1997-01-09	1997-01-16
8	10423	1997-02-06	1997-02-24
9	10427	1997-02-24	1997-03-03
10	10433	1997-03-03	1997-03-04

例題3の解答例

PostgreSQL

```
SELECT order_id , required_date, shipped_date FROM orders
WHERE (required_date - shipped_date) < 0 ;
```

SQLite (不等式を利用)

```
SELECT order_id , required_date, shipped_date FROM orders
WHERE required_date < shipped_date;
```

SQLite (ユリウス日を利用)

```
SELECT order_id , required_date, shipped_date FROM orders
WHERE julianday(required_date) - julianday(shipped_date) < 0;
```

最終課題 問6

- (1) まだ出荷されていない注文の総数を求めるSQL文を作成せよ（注：出荷されていない注文のshipped_dateはnullになっている）
- (2) 顧客から要望された届け日に間に合わなかった注文の総数を求めるSQL文を作成せよ
- (3) 注文番号と、注文から出荷までの日数（注文のあった日付と、出荷された日付の差とし、別名をshipping_periodとする）の組を、注文から出荷までの日数の大きい順で求めるSQL文を作成せよ。ただし、結果はカラムの並べ替えを行わずに、表示の行数は10に限定せよ（注：出荷されていない注文を除外することが必要である）
- (4) 顧客毎の売上の合計を求めたい。それは、例題1で求めた「注文と製品毎の注文金額」を顧客毎に合計したものとする。顧客ID（customer_id）と顧客毎の売上の合計（別名customer_salesとする）の2つの組を求めるSQL文を作成せよ。ただし、結果はカラムの並べ替えを行わずに、表示の行数は10に限定せよ

最終課題 問6(1)(2)(3)の検索結果例

(1)

	count bigint
1	21

(2)

	count bigint
1	37

(3)

	order_id [PK] smallint	shipping_period integer
1	10777	37
2	10660	37
3	10427	35
4	10924	35
5	10545	35
6	10380	35
7	10593	35
8	10309	34
9	10705	34
10	10709	34

最終課題 問6(4)の検索結果例

	 customer_id character 	customer_sales double precision 
1	TOMSP	4778
2	LONEP	4258
3	OLDWO	15178
4	WARTH	15650
5	MAGAA	7176
6	QUEEN	25718
7	VINET	1481
8	ANTON	7023
9	MORGK	5042
10	GOURL	8414

最終課題 問6

(5) 顧客毎の発注金額の状況を調査したい。そのため、例題2で求めた注文毎の注文金額と、注文毎の注文金額をさらに顧客毎に平均をとった金額（顧客平均注文金額と呼ぶ）を並べて表示したい。注文テーブル（orders）、注文詳細テーブル（order_details）を用い、注文番号（order_idカラム）、顧客ID（customer_idカラム）、注文毎の注文金額（別名をorder_priceとする）、顧客平均注文金額（別名をcustomer_avg_priceとする。結果はround関数を用いて整数にせよ）の4つの組を求めるSQL文をウィンドウ関数を利用し作成せよ。ただし、結果はカラムの並べ替えを行わずに、表示の行数は10に限定せよ。また、カラムの表示順は次ページの検索結果例と同じとすること

最終課題 問6(5)の検索結果例

	 order_id [PK] smallint 	customer_id character 	order_price double precision 	customer_avg_price double precision 
1	10643	ALFKI	815	712
2	10692	ALFKI	878	712
3	10702	ALFKI	330	712
4	10835	ALFKI	846	712
5	10952	ALFKI	471	712
6	11011	ALFKI	933	712
7	10625	ANATR	480	351
8	10926	ANATR	514	351
9	10759	ANATR	320	351
10	10308	ANATR	89	351